

Respostas da Atividade de Revisão - Integração e Entrega Contínua

Tema: Projeto INPE – Aplicativo de Monitoramento e Comunicação de Eventos Climáticos e Ambientais Críticos

PARTE I – QUESTÕES DE MÚLTIPLA ESCOLHA

1. **Questão 1:** c) Uma parcela significativa do código pode não estar sendo validada pelos testes.
2. **Questão 2:** b) detectar problemas antes da implantação.
3. **Questão 3:** c) identificar vulnerabilidades em dependências.
4. **Questão 4:** b) a cobertura dos testes precisa ser ampliada.
5. **Questão 5:** c) Fornecem indicadores visuais sobre a saúde do projeto.

PARTE II – QUESTÕES DISCURSIVAS

Questão 6

Teste automatizado: São scripts escritos para validar se o sistema ou partes dele funcionam conforme o esperado. Exemplo no INPE: um teste que verifica se a função que emite o alerta de queimada é disparada corretamente ao receber um dado positivo do sensor.

Cobertura de testes: É a métrica que indica qual a porcentagem do código-fonte que é executada pelos testes automatizados. Exemplo no INPE: garantir que os testes passam tanto pelos alertas de enchentes quanto de queimadas, e não apenas por uma das rotas.

Vulnerabilidade de software: Uma falha de segurança no código ou em dependências que pode ser explorada. Exemplo no INPE: uma falha em uma biblioteca de banco de dados (como PostgreSQL) ou Node.js que permite vazamento de dados dos usuários cadastrados no aplicativo.

Questão 7

Essa situação é um risco porque, embora os testes atuais passem com sucesso, 60% do código do aplicativo (que lida com alertas críticos para a população) não está sendo testado. Pode haver falhas críticas nessas áreas não cobertas, como falha no envio em massa de SMS, que não seriam detectadas pelo pipeline antes de ir para produção.

Questão 8

Recomendo que a equipe priorize a criação de testes unitários para a camada de **Utils**. A

justificativa é que funções utilitárias costumam ser compartilhadas e reutilizadas por todo o sistema (como formatação de datas de eventos climáticos ou validação de coordenadas). Uma falha não mapeada na pasta Utils pode gerar problemas em cadeia afetando os Controllers e Services que dependem dela.

Questão 9

Uma alta cobertura significa apenas que o código foi executado (passou por ele) durante a execução dos testes, mas não garante que todas as regras de negócio foram validadas corretamente (falta de asserts/expectations adequados). Além disso, defeitos em produção podem surgir por problemas de infraestrutura, integrações externas (como a própria API do INPE) ou configurações de ambiente que não são pegadas pelos testes de código.

Questão 10

O que faz: Executa a suíte de testes automatizados definida no script "test" do arquivo `package.json` em projetos Node.js.

Quando é executado: Em um pipeline CI/CD, é executado automaticamente em etapas de Build/Test, sempre que há um novo *commit* ou *Pull Request*.

Qual sua importância: Garante que as alterações recentes não quebraram as funcionalidades existentes do projeto do INPE, validando a integridade do sistema antes que o código siga para as próximas etapas (como o deploy).

Questão 11

Não, a decisão é inadequada. Vulnerabilidades classificadas como *High* (Altas) pelo `npm audit` representam sérias brechas de segurança. Considerando que o sistema do INPE envia alertas para a população, explorar essas falhas pode causar interrupções de serviço ou manipulação dos alertas (enviar alertas falsos). O deploy deveria ser interrompido (*broken build*) até que a equipe aplique a correção das dependências.

Questão 12

Vulnerabilidade em dependências de software ocorre quando bibliotecas ou pacotes de terceiros (usados no projeto) possuem falhas de segurança conhecidas. Como não controlamos o código-fonte original desses pacotes, ficamos suscetíveis. **Exemplo no INPE:** O aplicativo utiliza uma biblioteca externa em Node.js para converter os dados geográficos e de raster de risco de fogo em JSON. Se essa biblioteca tiver uma falha de Execução Remota de Código (RCE), um invasor pode enviar um payload malicioso via API e tomar o controle do servidor do aplicativo.

Questão 13

Não. Um resultado de 97% indica uma excelente cobertura de código, mas não garante a ausência de erros. Problemas de lógica onde o resultado validado no teste está incorreto de acordo com os requisitos (falso positivo), além de falhas de usabilidade, falhas de integração com serviços externos ou indisponibilidade de banco de dados não são detectados apenas por testes de cobertura de unidade.

Questão 14

Os relatórios de cobertura fornecem uma métrica objetiva e quantitativa sobre a qualidade das entregas da equipe. Se o gerente perceber que uma nova funcionalidade de alerta está sendo enviada para deploy com a cobertura caindo drasticamente, ele pode tomar a decisão de bloquear o deploy, exigindo que a equipe aumente a cobertura para mitigar riscos de incidentes no ambiente de produção.

Questão 15

A recomendação é **bloquear e cancelar o deploy** imediatamente. A justificativa técnica é que, embora a funcionalidade aparente estar correta (100% de testes passando com boa cobertura de 86%), as 5 vulnerabilidades de nível Alto representam um risco de segurança inaceitável. A equipe deve resolver as falhas de segurança nas dependências (usando

`npm audit fix`

, por exemplo) antes de prosseguir.

Questão 16

Os testes automatizados são a base que faz a Integração Contínua (CI) funcionar de verdade. A IC propõe a integração frequente de código no repositório principal. Para que isso ocorra com segurança, os testes automatizados são disparados automaticamente a cada integração, atuando como uma barreira que garante que o novo código funciona e não introduz regressões no projeto principal.

Questão 17

Porque se trata de um dado gerado diretamente por usuários externos, o que traz alta variabilidade e risco. A estratégia rigorosa de testes automatizados (testes de estresse, sanitização de inputs) é necessária para evitar ataques de injeção no banco de dados, garantir que o sistema não caia em picos de denúncias simultâneas (escalabilidade) e assegurar que as validações barrem dados maliciosos antes que isso polua a base oficial de monitoramento do INPE.

Questão 18

- **Build:** Indica se a compilação ou empacotamento do projeto foi concluído com sucesso.
- **Tests:** Mostra visualmente se todos os testes automatizados da aplicação foram executados sem erros.
- **Coverage:** Exibe a porcentagem do código-fonte que foi coberta pelos testes realizados.
- **Vulnerabilities:** Apresenta a quantidade e o nível de falhas de segurança conhecidas encontradas nas dependências do projeto.

Questão 19

A afirmação está incorreta. A revisão de código (*Code Review*) analisa aspectos que os testes automatizados muitas vezes ignoram: a arquitetura do código, boas práticas, legibilidade, padronização, regras de negócio complexas e otimização. A automação testa a execução mecânica, mas a revisão humana garante a manutenção a longo prazo e a qualidade estrutural da aplicação.

Questão 20

1. **Status do Build:** Deve estar concluído com sucesso, garantindo que a aplicação integra corretamente.
2. **Resultados de Testes:** 100% dos testes devem passar para assegurar a confiabilidade funcional.
3. **Vulnerabilidades de Segurança (ex: npm audit):** Ausência de vulnerabilidades Críticas ou Altas para proteger os dados sensíveis da população.
4. **Cobertura de Testes (Coverage):** Deve atender à meta estipulada pelo projeto para garantir validação abrangente do código.

Esses indicadores atuam em conjunto, garantindo a estabilidade e segurança necessárias para um software crítico de alertas ambientais.